ENGINEERING SIMULATION REPORT / V1.0

20-Axis Servo Control, Synchronization & Safety

Technical Implementation and Experimental Evaluation

C++17 reference simulator with heterogeneous joint dynamics, discrete PID, quintic motion profiles, delayed command delivery, fault injection, heartbeats and watchdogs.

20	1 ms	4
Joint models	Simulation step	Executed scenarios

Scope: Educational software simulation and reproducibility report
Evidence: Supplied C++ source, scenario CSV logs, event traces and plots
Date: 28 October 2026 | **Safety:** Not validated or certified for deployment on real robots

EXECUTIVE SUMMARY

01. Executive summary

This report documents a reproducible **20-axis heterogeneous servo simulation** implemented in C++17. It evaluates how a single trajectory clock, independent joint PID controllers, model-based feedforward, delayed command delivery, a supervisory state machine and simulated local drive watchdogs behave in four deterministic scenarios. The results and figures originate from the project source and saved CSV/event logs, not from a physical robot.

20 / 20
Joints reaching the prescribed terminal tolerance (nominal)

0.000406
Peak nominal absolute tracking error (rad, rounded)

128 ms
Spread of measured terminal arrival times

The scenario suite includes (i) nominal motion, (ii) a mechanical jam of joint 8 at 1.450 s, (iii) central controller heartbeat interruption, and (iv) heartbeat interruption for drive 13. The three fault cases transition to a simulated group-wide stop. The nominal scenario also includes a wall-clock timing run: target period 1 ms, measured P99 wakeup lateness 233.855 microseconds and maximum lateness 3854.283 microseconds in the recorded environment.

> **Scope boundary**
>
> The implementation does **not** control actual hardware. Its SAFE_STOP and ESTOP labels describe simulation states, not safety-certified STO, SS1, SS2, emergency-stop circuits or mechanical braking. Its numerical joint parameters were configured synthetically rather than identified from a specific motor.

Key findings

- All joints share the same planned motion interval (0.250 to 3.250 s), but actual terminal tolerance entry is spread across 128 ms. Planned-time synchronization is not identical to measured arrival-time synchronization.
- A jam is detected from sustained position-following error 356 ms after injection. This is a relevant weakness of error-only detection, not evidence of rapid collision detection.
- Both heartbeat-loss tests enter SAFE_STOP at 1.490 s and reach the STOPPED simulation state about 172-175 ms later. Timeouts are measured from the last healthy heartbeat, not the instant of injected failure.
- Linux/WSL user-space wakeup jitter has occasional millisecond-scale excursions; therefore the recorded experiment cannot establish hard real-time performance.

Reading guide

Part	Chapters	What it answers
Architecture and plant	02-04	Requirements, data flow, 20 distinct joint models
Motion control	05-07	Quintic synchronization, discrete PID, delays and clocks
Protection and evidence	08-11	Heartbeat/watchdog logic, fault injection, plots, jitter
Reproducibility	12-14	Commands, limitations, roadmap, verification checklist

REQUIREMENTS AND SCOPE

02. Requirements, acceptance criteria and assumptions

The project is designed as a learning and diagnostic testbed for questions encountered in robotic servo engineering: how to keep many motors on one timebase, handle varied inertias, model delayed commands, identify stopped or jammed joints and react to missing control signals.

Objective	Implemented mechanism	Evidence and checks

Heterogeneous axes	Different inertia, damping, friction and actuator limits	`make_config()` and joint response logs
Shared motion timing	One 3.0 s quintic progression for 20 targets	Planned terminal time and terminal-entry events
Closed-loop tracking	PID + dynamic feedforward + torque/torque-slew caps	Per-joint error, velocity and torque CSV channels
Network latency	20 fixed-capacity delay lines and message sequences	Queued delivery logic and reproducible latency schedule
Abnormal conditions	Jam, controller heartbeat loss, drive heartbeat loss	Events, stop reasons and simulated speed decay
Timing characterization	`steady_clock` + `sleep_until` in realtime mode	Wakeup lateness and inter-wakeup period error

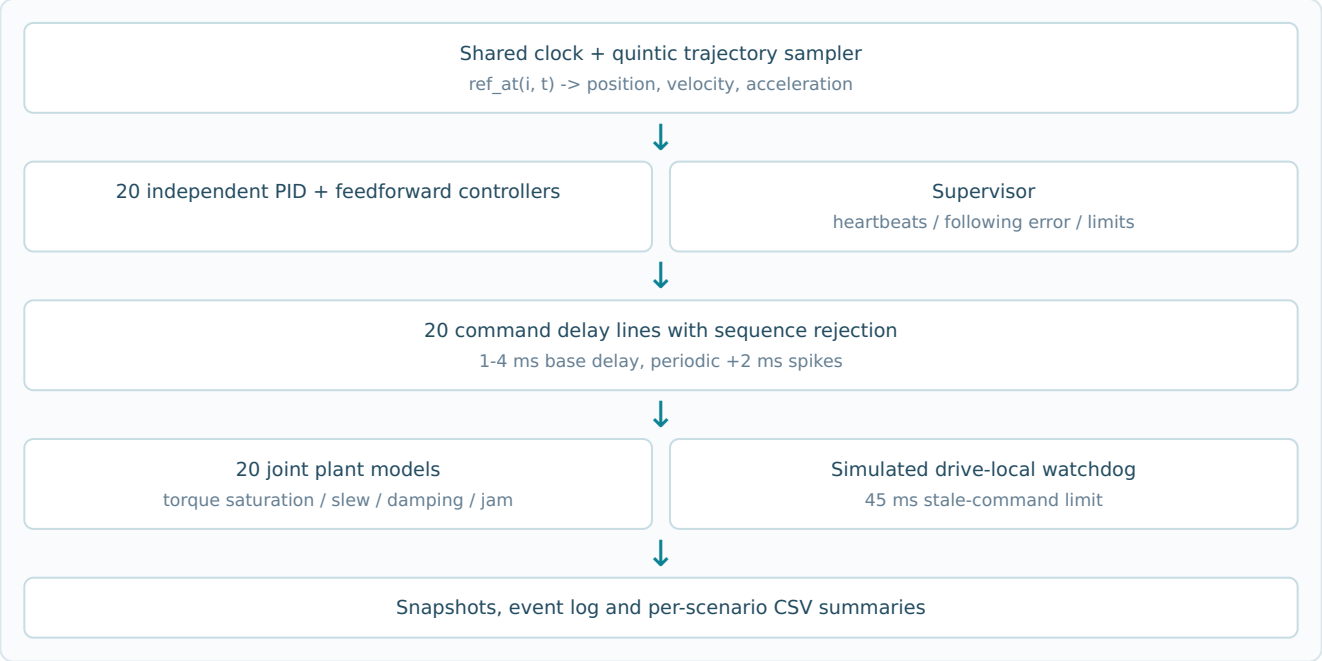
Explicit simplifying assumptions

- Each joint is an independent one-degree-of-freedom rotary inertia; there is no robot-wide mass matrix or link coupling.
- Gravity, gear elasticity, actual motor electrical dynamics, encoder noise, thermal behavior and physical impacts are not represented.
- Drive current is not simulated. The reported *torque* is the applied model torque, not measured phase current.
- Network delay is a deterministic software queue, not a measurement of CAN, EtherCAT or any real bus.
- The code is a single simulation process. The simulated local-drive watchdog is not independent hardware and therefore cannot provide fail-operational protection against total process or host failure.

SOFTWARE ARCHITECTURE

03. System architecture and main cycle

The C++ program executes one global time-stepped loop at **DT = 0.001 s**. At each tick, it samples 20 desired states at exactly the same simulation time, computes torque commands, inserts them into per-axis delay lines, consumes eligible packets, evaluates safety conditions, integrates plant dynamics and records a snapshot. This reduces scheduling ambiguity during algorithm verification, but is not a multi-threaded real-time production architecture.



Module-to-code mapping

Code symbol	Main responsibility

`make_config()`	Create 20 reproducible, heterogeneous joint parameter sets
`ref_at()`	Return desired position, velocity and acceleration at shared time t
`PID::update()`	Compute bounded torque command with feedforward and anti-windup
`DelayLine`	Model packet latency and discard obsolete sequence numbers
`Joint::simulate()`	Enforce torque slew and integrate rotary dynamics
`run()`	Orchestrate synchronization, supervisor, faults, logging and metrics

PLANT MODEL

04. Joint dynamics and heterogeneous parameters

4.1 Continuous-time model

$$J_i \ddot{q}_i + D_i \dot{q}_i + \tau_{c,i} \tanh(\dot{q}_i / 0.025) = \tau_{applied,i}$$

Here J denotes equivalent rotational inertia (kg m²), D is viscous damping (N m s/rad), and τ_c is a smoothed Coulomb-friction amplitude (N m). The tanh term avoids a discontinuity at zero velocity. These are *physical model coefficients*, not PID gains or virtual impedance gains.

After applying torque amplitude and slew-rate limits, the simulator uses semi-implicit Euler integration: velocity is updated from acceleration and position is updated using the new velocity. Physical position and speed are not silently clamped; exceeding a configured limit is instead handled by the supervisor.

$$a_k = [\ \tau_k - D\ v_k - \tau_c \tanh(v_k/0.025)\ ] / J$$
$$v_{k+1} = v_k + a_k\ DT \quad ; \quad q_{k+1} = q_k + v_{k+1}\ DT$$

4.2 Parameter ranges

Parameter	Configured range	Units
Inertia J	0.012-0.031	kg m²
Viscous damping D	0.027-0.043	N m s/rad
Smoothed Coulomb friction	0.0025-0.0040	N m
Maximum torque	0.70-1.03	N m
Maximum torque slew	20-26	N m/s
Maximum joint speed	1.00-1.24	rad/s
Joint position protection	+/-1.7	rad

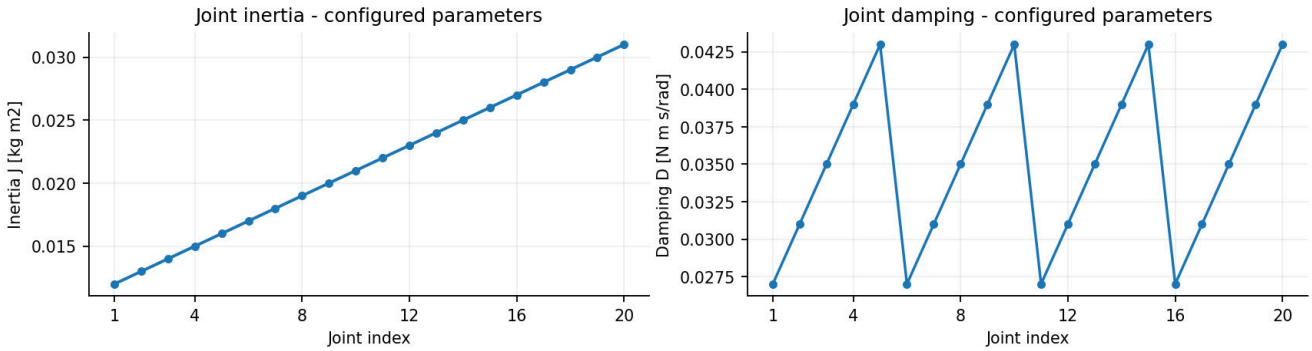


Figure 4-1. Joint inertias and viscous damping values taken directly from the simulator configuration (synthetic, not identified).

4.3 Model limitations

A jam is represented as zero velocity with frozen position; the motor torque may still build up. This idealization can demonstrate a tracking-error fault, but does not predict motor current, gear stress, winding temperature, contact forces or damage.

MOTION PLANNING

05. Quintic interpolation and coordinated motion

5.1 One timeline, different target angles

Motion begins at **0.250 s** and lasts **3.0 s**, so every reference trajectory ends at **3.250 s**. Joint targets alternate in sign; their absolute magnitude increases from 0.200 to 0.637 rad. All joints share the normalized quintic progression, but do not share the same angular speed.

$$r = (t - 0.250)/3.0 \quad ; \quad s(r) = 10r^3 - 15r^4 + 6r^5$$
$$q_{d,i}(t) = q_{0,i} + \Delta q_i \, s(r)$$
$$v_{d,i}(t) = (\Delta q_i/T) \, s'(r) \quad ; \quad a_{d,i}(t) = (\Delta q_i/T^2) \, s''(r)$$

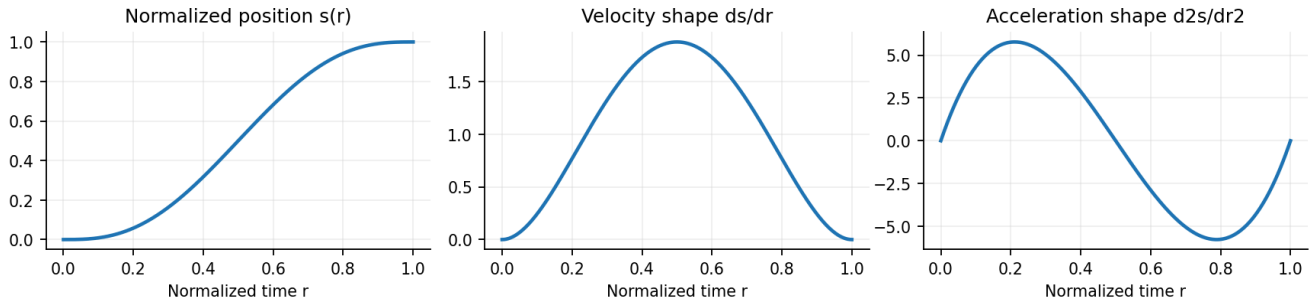


Figure 5-1. Position, velocity and acceleration profiles of the common normalized quintic interpolant.

5.2 Feasibility of the shared duration

Choosing one duration T is only valid if all joint limits allow it. For this quintic profile, the maximum normalized velocity is 1.875. A necessary velocity condition is:

$$T \geq \max_i \{ \, 1.875 \, |\Delta q_i| \, / \, v_{max,i} \, \}$$

Acceleration, jerk, torque, temperature and hardware-specific limits must also be checked. This demonstrator fixes T = 3.0 s; it does not solve a constrained minimum-time retiming problem or implement online replanning.

5.3 The distinction between reference completion and actual arrival

Measured arrival requires **|q - q_target| < 0.006 rad** and **|v| < 0.014 rad/s** continuously for 40 ticks. Detection begins only within the final 0.30 s before the planned end. The stored arrival timestamp is the beginning of the qualifying 40 ms window. Because the tolerances are nonzero, a joint can qualify before the trajectory has completely finished.

> **How to interpret the 128 ms spread**
>
> The measured 128 ms is the range between earliest and latest qualifying terminal-entry timestamps under this particular criterion. It does **not** mean commands were dispatched 128 ms apart or that the reference trajectories ended at different times.

CONTROL LAW

06. Discrete PID, model feedforward and actuator limiting

6.1 Control equation

$$e[k] = q_d[k] - q[k]$$
$$\tau_{cmd}[k] = K_p e[k] + K_i I[k] + K_d(v_d[k] - v[k]) + \tau_{ff}[k]$$

The derivative-like term uses desired minus measured velocity instead of directly differencing position error. The feedforward term is calculated from the same simplified joint model:

$$\tau_{ff} = J\, a_d + D\, v_d + \tau_c \tanh(v_d/0.025)$$

Gains are initialized from a nominal natural frequency $\omega_n = 22$ rad/s and damping ratio $\zeta = 0.90$:

$$K_p = J\, \omega_n^2 \quad ; \quad K_d = \max(0, 2\zeta\omega_n J - D) \quad ; \quad K_i = 1.5$$

This is a convenient tuning heuristic for the model, not a robustness proof. There is no experimentally estimated plant here; model and feedforward are deliberately closely matched.

6.2 Anti-windup and saturation

The integral candidate is clamped to [-0.15, +0.15]. If accepting that candidate would increase output saturation in the direction of the present position error, the integral is held. The final torque is bounded by the joint torque limit; the plant additionally applies a maximum change in applied torque per tick.

```
candidate = clamp(integral + error*DT, -0.15, 0.15);
trial = pd_plus_feedforward + Ki*candidate;
if (!pushing_further_into_saturation)
    integral = candidate;
command = clamp(pd_plus_feedforward + Ki*integral,
                -max_torque, +max_torque);
```

PID integrators are reset on fault-triggered entry into SAFE_STOP to prevent accumulated tracking errors from appearing as unexpected control actions after a hypothetical restart. A real controller additionally needs explicit mode-transition semantics and reinitialization checks.

6.3 Servo-loop boundary

This simulator assumes a torque-command interface and perfect velocity availability. A real commercial drive may already contain its own cascaded current, speed and position loops. One must establish whether the available API controls position, velocity, current or torque before placing another controller above it.

TIMING AND COMMUNICATION

07. Delay queues, sequence numbers and clock synchronization

7.1 Command packet lifecycle

Each joint has a fixed array of 32 pending packets, with fields for delivery tick, sequence number and torque. Commands experience 1-4 ms base latency and occasionally an additional 2 ms delay, for a modeled maximum of 6 ms. At every tick, due packets are inspected, and only a packet newer than the last accepted sequence can update the applied command.

$$\text{delay}_{\text{ticks}} = 1 + (\text{joint_index} \bmod 4) + \text{periodic_extra_2_ticks}$$

Overflow increments a counter. In this version overflow is *not* connected to a supervisory fault. A production implementation should also enforce deadlines, capacity policies and visibility of dropped, duplicated or late commands.

7.2 Synchronization has several meanings

Concept	Meaning	What this simulator proves
Reference-time synchronization	All axes sample one planned trajectory time	Implemented
Feedback-loop scheduling	Computations use one simulated 1 ms step	Implemented in one process
Hardware execution alignment	Drives latch/apply commands on a shared device clock	**Not implemented**
Coordinated physical arrival	All axes meet a measured process tolerance in a prescribed window	Measured only under a specific tolerance definition

A CAN network transmits individual frames serially under arbitration; hardware simultaneity requires appropriate drive behavior such as preloaded setpoints plus a synchronization trigger. EtherCAT Distributed Clocks are one approach for synchronizing compatible devices. Neither bus is physically implemented in this project.

7.3 Recommended future packet interface

Field	Reason
`sequence`	Reject duplicates and stale commands
`timestamp_sent`	Measure upstream communication latency
`execution_time`	Separate receipt time from synchronized application time
`heartbeat_counter`	Detect stale sender state rather than merely link activity
`mode, health, fault_flags`	Prevent unsafe or unintended command transitions

FAULT CONTAINMENT

08. Heartbeats, watchdogs and supervisory state machine

8.1 Distinguishing three mechanisms

Mechanism	Configured behavior	Limitation
Controller heartbeat	Updated every 10 ms; age > 50 ms triggers group stop	Heartbeat freshness alone does not prove fresh motion commands
Drive heartbeat	Updated every 10 ms per drive; age > 50 ms triggers group stop	Modeled in same process, not an independently verified drive interface
Local command watchdog	If no newly accepted command for > 45 ms, invoke local damping-brake model	Simulated in-process; no independent embedded execution or safety rating

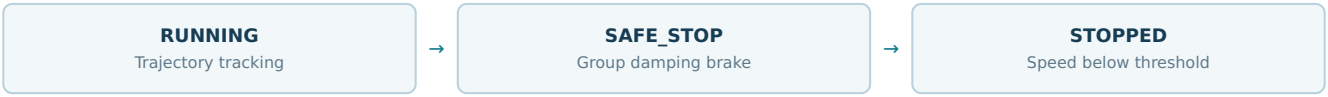
A sender may still produce healthy heartbeats while its trajectory generator has stopped. Therefore heartbeat age and accepted-command age are distinct signals. In this simulation, stale command behavior is additionally monitored at drive-model level.

8.2 Other monitored conditions

- Following error greater than 0.065 rad for 60 consecutive 1 ms ticks, once motion has progressed past the initial transient window.
- Joint position or velocity exceeding the configured maximum.
- Nonfinite joint position or velocity measurements.

- Unacknowledged drive heartbeat or controller heartbeat within the configured age threshold.

8.3 Simulation state transitions



Once a fault triggers, PID states are reset and all joints enter the simulated braking behavior. The STOPPED state requires maximum absolute velocity below 0.006 rad/s for 50 consecutive ticks. Failure to satisfy that within 0.8 s results in the software state label ESTOP.

> **Crucial safety distinction**
>
> The simulated brake is a velocity-proportional torque command (approximately -7.5 v, with limits). It is **not** a safety-rated stop and does not model load holding. On a gravity-loaded robot, abruptly removing torque can cause falling or uncontrolled movement; a real design must follow the drive and robot manufacturer safety architecture.

8.4 Failure of the monitor itself

All safety logic currently runs inside the same process and timed loop. If that host or loop stops entirely, the software supervisor also stops. This is the core reason to place the ultimate timeout enforcement in an independently executing drive/controller or properly engineered hardware safety chain in a real system.

EXPERIMENT DESIGN

09. Fault injection and measurement methodology

All four runs use the same 20-joint configuration and 1 ms discrete simulation step over 4.5 s. The nominal run used `--realtime` to collect wall-clock scheduling data; fault runs used accelerated virtual time and intentionally do not report fabricated scheduling jitter.

Scenario	Injected event	Expected evidence
`nominal`	No fault injection	All 20 reach terminal tolerance; no safety trigger
`jam`	Joint 8 position freezes at 1.450 s	Following error persists; group transitions to stop
`controller_heartbeat`	Controller heartbeat stops at 1.450 s	Controller-heartbeat timeout
`drive_heartbeat`	Drive 13 heartbeat stops at 1.450 s	Drive-13 heartbeat timeout

Available data products

For each scenario, the simulator writes a sample-by-sample CSV, a discrete event CSV and a summary text file. The plotting script renders tracking error, braking curves, timing behavior and arrival charts. All values presented here are from those saved files; this report does not run a different dataset.

RESULTS

10. Results and interpretation

10.1 Scenario-level outcomes

Run	Trigger (s)	STOPPED (s)	Time since injection to trigger	Final
Nominal	None	n/a	n/a	RUNNING, all arrived
Joint 8 jam	1.806	1.981	356 ms	STOPPED
Controller heartbeat	1.490	1.662	40 ms	STOPPED

Drive 13 heartbeat	1.490	1.665	40 ms	STOPPED

The heartbeats are normally emitted at 10 ms intervals. The final heartbeat before injection is at about 1.440 s, so timeout at 1.490 s corresponds to ~50 ms since the last valid heartbeat even though it is only 40 ms after the injection time.

10.2 Nominal tracking error

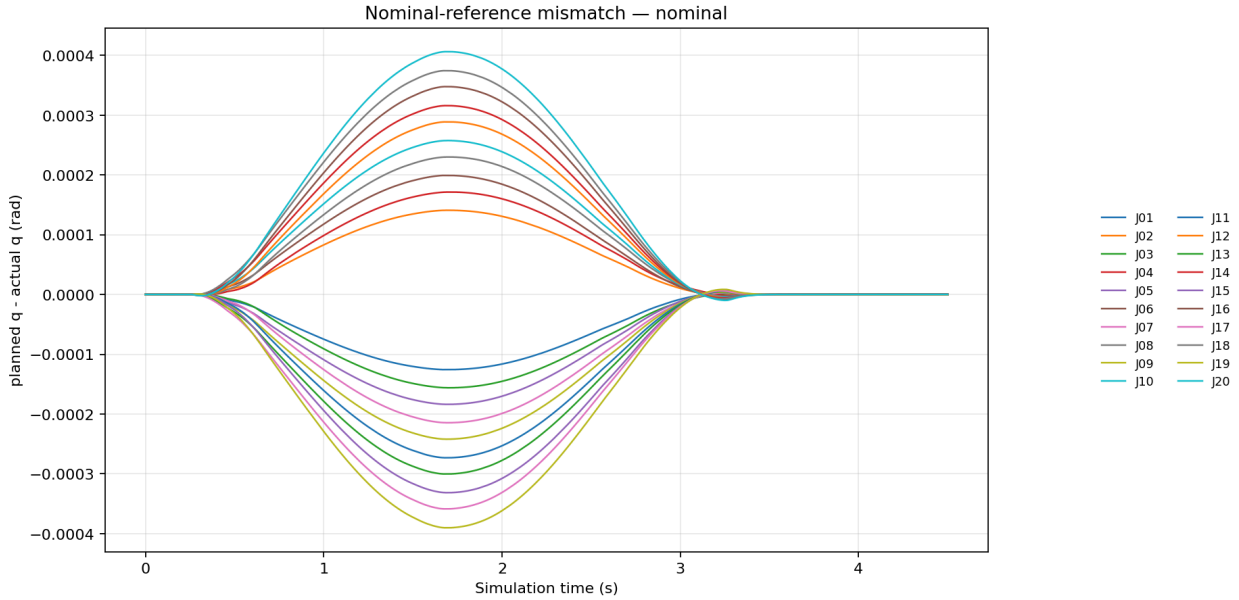


Figure 10-1. Tracking error of all 20 joints during nominal execution. Peak absolute error over the full saved CSV: 0.0004063 rad, approximately 0.0233 degrees.

All 20 traces track their reference closely in this near-model-matched setting. The result is evidence that the software implementation functions under its configured assumptions, not a real actuator accuracy claim or proof of robustness to parameter mismatch.

10.3 Measured terminal arrival

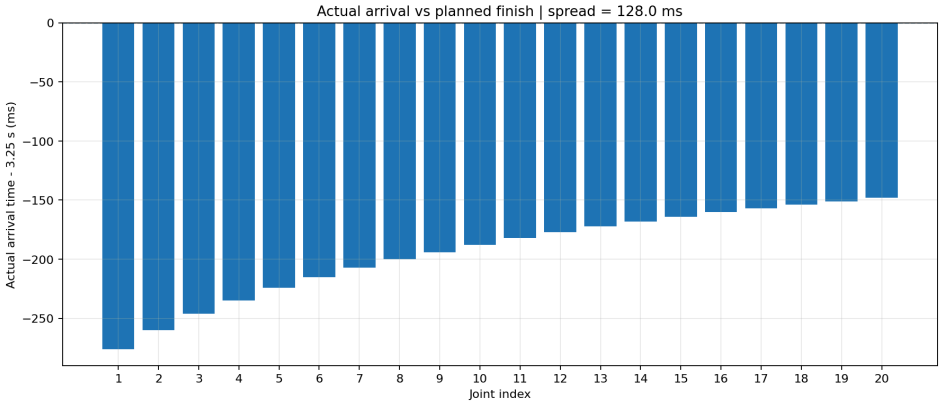


Figure 10-2. Per-joint measured stable terminal-entry time relative to the common reference end at 3.250 s. Earliest-to-latest spread: 128 ms.

All joints qualify under the configured terminal tolerance before the nominal 3.250 s end. The earliest qualifying time is 2.974 s and the latest 3.102 s, consistent with the use of nonzero speed/position tolerances. A stronger coordination metric would assess normalized trajectory progress errors throughout the move.

10.4 Joint 8 jam

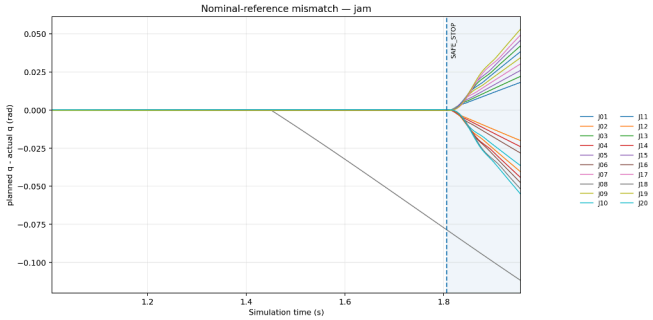


Figure 10-3. Joint 8 locks at 1.450 s. Its continuing reference motion leads to rising following error and a protection trigger at 1.806 s.

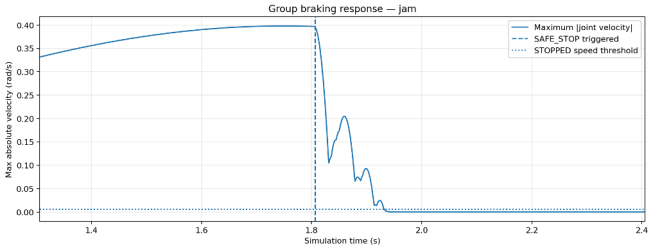


Figure 10-4. Maximum joint speed during the subsequent group damping-brake response.

The 356 ms delay reflects the threshold plus the time needed for reference/actual position discrepancy to grow. This test motivates faster independent measurements: velocity residuals, estimated versus measured current or torque, saturation evidence and contact/ collision diagnostics.

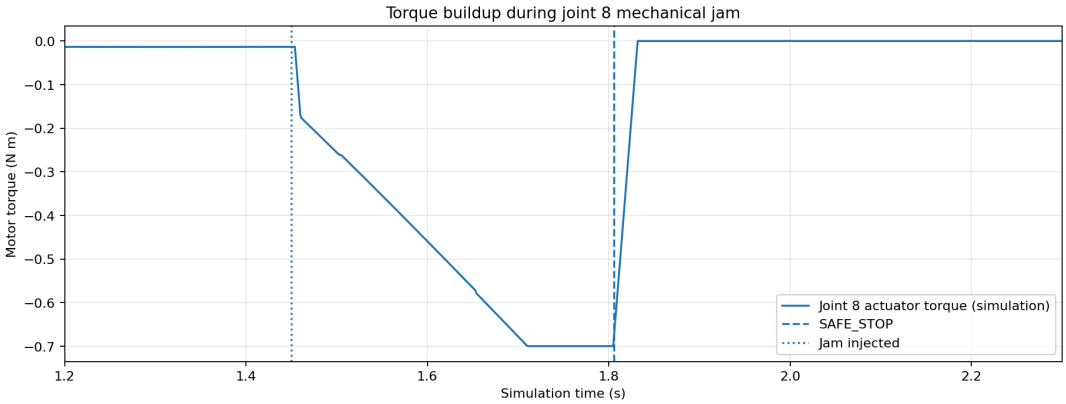


Figure 10-5. Simulated joint 8 motor torque during the jam. This is neither measured electrical current nor an estimate of gearbox structural stress.

10.5 Controller and drive heartbeat interruptions

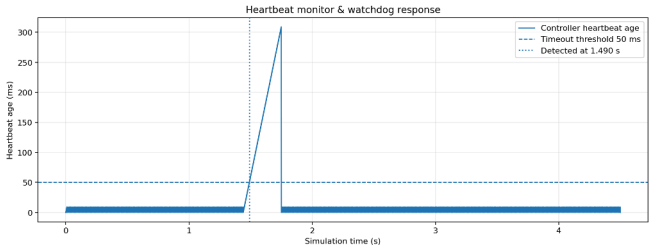


Figure 10-6. Controller heartbeat age crosses the configured threshold before the group enters software SAFE_STOP.

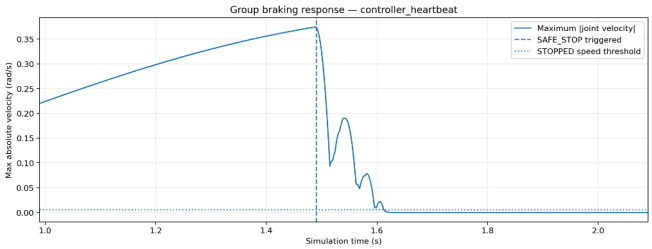


Figure 10-7. Group speed reduction following simulated controller heartbeat timeout.

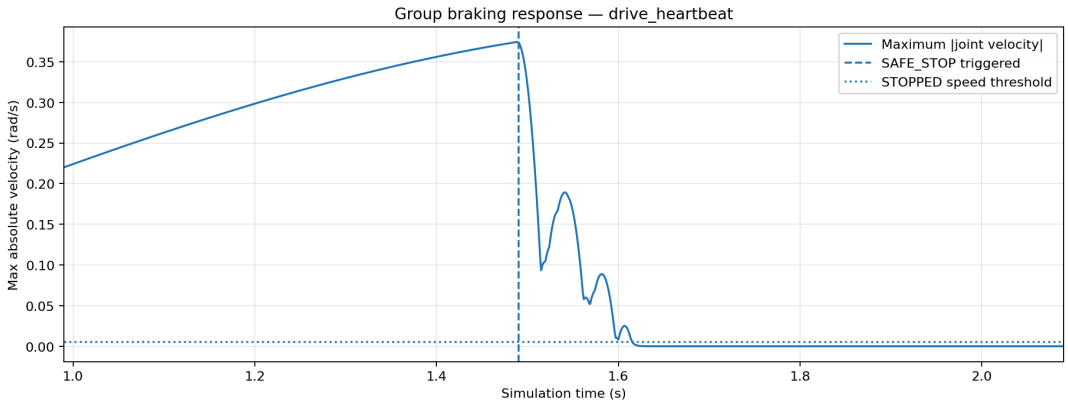


Figure 10-8. Group damping-brake response following simulated heartbeat loss at drive 13.

The two heartbeat cases show that the code records distinct stop reasons. They do not validate that an actual fieldbus driver detects a physically disconnected node, nor that any real device independently enforces the watchdog.

10.6 Why error curves grow after some stops

Saved `q_ref_i` signals continue to reflect the *original planned trajectory* after safety triggering for diagnostic purposes. The plant is no longer tasked with tracking that reference. Therefore a large logged post-stop reference minus actual difference is not automatically a failed brake. Braking quality must instead be assessed from velocity, applied torque and state-transition timing.

TIMING CHARACTERIZATION

11. Scheduler jitter and real-time claims

Only the nominal run used `--realtime`, executing `std::this_thread::sleep_until` against `std::chrono::steady_clock`. The software records both wakeup lateness relative to scheduled absolute time and error of successive wakeup intervals relative to 1000 microseconds.

$$\text{wakeup_lateness}_k = \text{wakeup}_k - \text{scheduled}_k$$
$$\text{period_error}_k = (\text{wakeup}_k - \text{wakeup}_{k-1}) - 1000\ \mu s$$

233.855 µs
Wakeup lateness P99

3854.283 µs
Maximum wakeup lateness

305.080 µs
Absolute period error P99

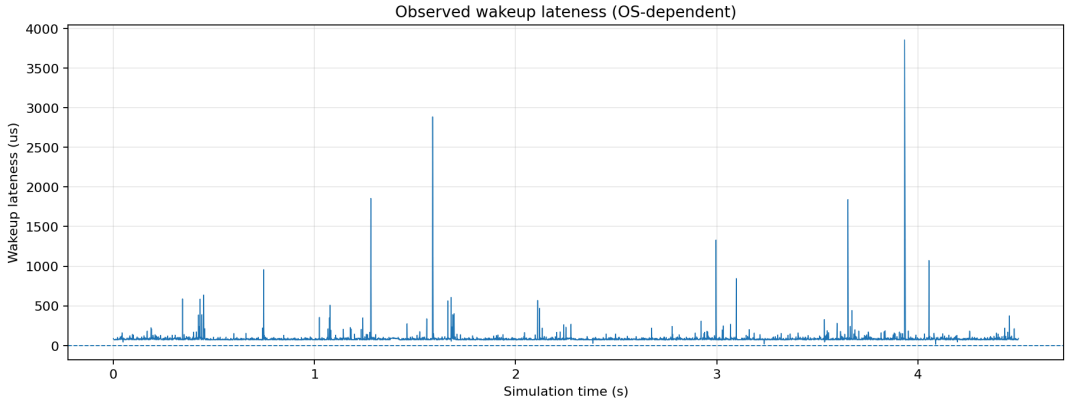


Figure 11-1. Observed wakeup lateness across the nominal 1 ms scheduled loop.

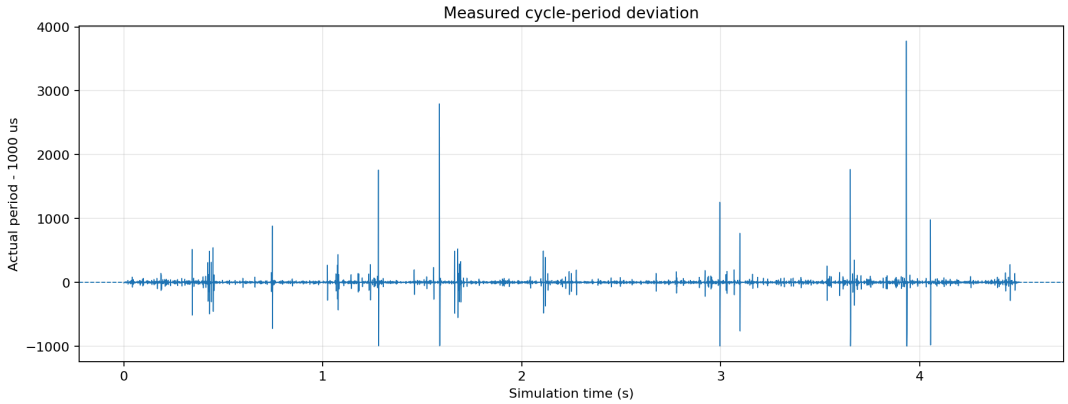


Figure 11-2. Inter-wakeup interval error. The saved run contains 9 wakeups more than 1 ms late; observed maximum absolute inter-wakeup period error was approximately 3.78 ms.

Hard real-time is not demonstrated

The configured target interval is 1 ms, but actual wakeups show multi-millisecond deviations. More fundamentally, the implementation does not measure complete control worst-case execution time, deadline miss rate under long-term load, or when hardware actuators physically latch a setpoint. It cannot establish a deterministic 1 kHz hardware servo.

What a stronger timing validation needs

- Separate target period, actual start/finish times, worst-case execution time, missed deadlines, and maximum wakeup delay.
- Run on the target controller with a suitable real-time scheduling environment; avoid unbounded allocation and blocking I/O in the control path.
- Keep CSV recording and plotting out of the hard real-time thread; use bounded preallocated inter-thread transfer.

- Check network device timestamps and actual command-latching jitter separately from host scheduling jitter.

REPRODUCTION

12. Build, reproduce and inspect the implementation

12.1 Repository layout

```
servo20-control-safety/
  CMakeLists.txt
  src/main.cpp
  tools/plot_results.py
  tools/check_results.py
  run_all.sh
  results/             # Example scenario CSVs and event logs
  plots/               # Generated visual results
  reports/
    Servo20_Control_Safety_Technical_Report_CN.pdf
    Servo20_Control_Safety_Technical_Report_EN.pdf
  README.md            # English overview
  README_CN.md         # Chinese overview
```

12.2 Ubuntu / WSL instructions

```
sudo apt update
sudo apt install -y build-essential cmake python3-matplotlib
cmake -S . -B build
cmake --build build -j

./build/servo20 --scenario nominal --realtime --out results
./build/servo20 --scenario jam --out results
./build/servo20 --scenario controller_heartbeat --out results
./build/servo20 --scenario drive_heartbeat --out results

python3 tools/plot_results.py --input results --output plots
python3 tools/check_results.py
```

`bash run_all.sh` automates the full build, four scenarios, plot generation and regression checks. Repeating `--realtime` can yield different timing results depending on the scheduler and machine load. Do not compare accelerated virtual simulation to wall-clock latency metrics.

12.3 Suggested source reading order

Code	Engineering question
`ref_at()`	How are 20 independent targets assigned one motion timeline?
`Joint::simulate()`	What do inertia, damping, torque and a jam do to a joint response?
`PID::update()`	How do discrete feedback, feedforward and anti-windup work?
`DelayLine`	How are delayed and stale commands handled?
`run()`	How do fault detection, group-stop policy and logs interact?

ENGINEERING EVOLUTION

13. Gaps, risk assessment and development roadmap

13.1 What remains non-production-grade

Current approach	Gap	Next implementation
Single-thread main loop	Controller and supervisor can fail together	Independent supervisor / drive-local hardware enforcement

Decoupled rigid joints	No gravity or inter-axis dynamics	Coupled manipulator model and load holding constraints
Perfectly aligned feedforward model	Understates parameter uncertainty	Parameter perturbations, noise and identification tests
Synthetic command delays	No real arbitration, bus timing or device sync	CAN/EtherCAT test rig, execution timestamps
Position-error-only jam alarm	High detection delay for some faults	Current/torque residual and encoder integrity checks
Simulated damping stop	No demonstrated functional safety	Safety-rated drive interfaces and formal risk analysis

13.2 Five incremental milestones

1. **Reproducible benchmarking:** parameter export, random-seed control, configuration files, per-joint statistics and automated scenario evaluation.
2. **Real-time architecture:** separate control, trajectory planning and logging; use bounded preallocated queues and measure worst-case timing.
3. **Fault coverage:** inject encoder freezes/spikes, torque measurement errors, late packets, bus saturation, high friction and single-axis actuator shutdown.
4. **System identification:** implement chirp/PRBS experiments; fit J, D and friction with independent validation data, and test robustness to model mismatch.
5. **Hardware-in-the-loop:** map commanded setpoints to an actual drive, verify timestamp alignment and commission independent safety features with qualified personnel.

13.3 Safety priorities

Hazard class	Potential outcome	Mitigation class
Entire process freezes	Internal software monitors cease operating	Independent device watchdog and certified safety path
Joint jams under torque	Overload or damage	Independent current/torque limiting and residual diagnostics
Late or repeated commands	Stale or uncoordinated motion	Deadline enforcement and synchronized execution protocol
Gravity-loaded joint loses torque	Uncontrolled descent	Mechanical holding brake and manufacturer-defined safe stop

VERIFICATION AND SOURCES

14. Verification checklist, interview narrative and references

14.1 Evidence quality checklist

Test dimension	Useful metric	Status in this version
Tracking	Maximum/RMS error and actuator limit usage	Saved error, torque and velocity channels
Multi-axis coordination	Progress error, terminal spread, task-level tolerance	Terminal-entry spread only
Communication	Latency percentiles, reordering, packet-loss rate	Deterministic delays; no complete channel statistics
Fault detection	Trigger delay, stopping duration, false alarms	One deterministic trial per scenario
Real-time execution	WCET, deadline misses, long-duration P99.9	Wakeup lateness and period error only
Hardware safety	Certified stop, safe load hold, fault coverage	Not implemented or verified

14.2 An interview-ready technical summary

90-second description

I implemented a C++17 simulator of 20 heterogeneous servo axes, each with different inertia, damping, friction and torque limits. Every axis uses a discrete PID plus model feedforward while sharing a three-second quintic motion clock. To make the testbed more realistic, I added per-axis delayed command queues with sequence checks and simulated controller and drive heartbeats, local command watchdogs and a group-level stopping supervisor. Under the nominal synthetic model all 20 axes reached a specified terminal tolerance, with a maximum absolute position error of about 0.000406 rad. An injected jam of joint 8 was detected after 356 ms; controller and drive heartbeat interruptions triggered separate software stop reasons. The 1 ms wall-clock test showed millisecond-scale scheduling outliers, so I explicitly distinguish algorithmic synchronization, host scheduling and real hardware safety. My next step would be to improve fault-detection latency, separate runtime responsibilities and verify a genuine drive interface.

14.3 Primary artifacts and external background

1. **[S1] Implementation:** `src/main.cpp`, including plant dynamics, control law, delayed command queue and supervisor.
2. **[S2] Recorded evidence:** `results/` scenario CSVs, event traces and summary text; derived figures in `plots/`.
3. **[R1] ROS 2 Design - Real-time Background:** https://design.ros2.org/articles/realtime_background.html.
4. **[R2] Linux man-pages - clock_nanosleep(2):** https://man7.org/linux/man-pages/man2/clock_nanosleep.2.html.
5. **[R3] Beckhoff - EtherCAT technology:** https://www.beckhoff.com/en-en/products/i-o/ethercat/.
6. **[R4] Siemens - Safety Integrated commissioning manual:** https://support.industry.siemens.com/cs/attachments/109816998/MC_SI_commiss_man_0123_en-US.pdf.

Final assessment

The simulator demonstrates reproducible algorithm-level behavior for shared-time trajectory generation, heterogeneous PID tracking, bounded delays and fault-response logic. It does not provide evidence of actual actuator performance, independently functioning watchdog hardware, device-level synchronization or functional-safety compliance. It is a useful engineering training and hardware-in-the-loop starting point, not deployable robot safety software.

Figures and numerical metrics are from the project files supplied with this report. Re-running the simulator, particularly the wall-clock timing experiment, may change measured results. All thresholds are educational simulation settings.